A COLLEGE PROJECT REPORT ON

PathForge AI: Autonomous Multi-Agent Learning Path & Coding System

Skill Gap Diagnostics, Sequential Curriculum Design, Curated Resources,
Interactive Adaptive Assessment & NeetCode DSA Coding Arena using Python & CrewAI

Submitted in partial fulfillment of the requirements for the Course / Degree in
MASTER OF COMPUTER APPLICATIONS (MCA — SEMESTER III)

SUBMITTED BY

Name: Akhil Lehhan

Project ID: MCA-Sem3 · PathForge AI

Specialization: Multi-Agent AI & Full-Stack

SUBMITTED TO / GUIDED BY

Department of Computer Applications

Faculty Project Guide

Academic Session: 2026 – 2027

AUTONOMOUS AI PIPELINE

4 CrewAI Agents

Profiler, Curriculum, Resource & Assessment

LLM & TOKEN ENGINE

Dual Groq Keys · 900 MaxTok

qwen/qwen3.8-27b + Scoped Task Contexts

INTERACTIVE ASSESSMENT

7-Q Quiz + Auto-Grade

MCQ Radio Options, Code Editor & New Rounds

DSA PRACTICE ARENA

12 NeetCode Problems

6 REST Endpoints · Isolated 5s Code Sandbox

DEPARTMENT OF COMPUTER APPLICATIONS (MCA)
ACADEMIC YEAR 2026 – 2027

Certificate, Declaration & Acknowledgement

Academic Authenticity Statement & Project Credits

CERTIFICATE OF ORIGINALITY

- This is to certify that the project report entitled 'PathForge AI: Autonomous Multi-Agent Personalized Learning Path, Adaptive Assessment & DSA Coding System' is a bona fide record of independent project work carried out by Akhil Lehhan in partial fulfillment of the requirements for the Master of Computer Applications (MCA Semester III) curriculum during the academic year 2026–2027.
- The project has been implemented from scratch in Python and ES6 JavaScript (agents.py, tasks.py, crew.py, api.py, test_agents.py, and frontend/), integrating 4 autonomous CrewAI agents with Groq LLM inference, interactive quiz grading, and a live Python DSA test-case execution sandbox.
- Signature of Project Guide: _____ Signature of HOD / Examiner: _____

STUDENT'S DECLARATION

- I, Akhil Lehhan, hereby declare that the project work entitled 'PathForge AI: Autonomous Multi-Agent Personalized Learning Path, Adaptive Assessment & DSA Coding System' submitted by me is my original work completed during this academic semester.
- All multi-agent prompt workflows, dual Groq API key load-balancing, LiteLLM cache_breakpoint patches, interactive MCQ/Code quiz parsers, NeetCode-style DSA problem test harnesses, and Docker deployment configurations were built and executed by me.
- Place: India | Date: November 2026 Candidate Signature: Akhil Lehhan

ACKNOWLEDGEMENT

- First and foremost, I would like to express my sincere gratitude to my Project Guide and the faculty members of the Department of Computer Applications for their continuous guidance, constructive feedback, and encouragement throughout the creation of this project.
- I would also like to thank the open-source maintainers of CrewAI, FastAPI, LiteLLM, and Groq for providing the multi-agent orchestration framework and high-speed LPU inference APIs that powered this system.
- Finally, I am grateful to my peers and family for their support during the architecture design, debugging, UI engineering, and documentation phases of this project report.

Abstract & Table of Contents

Executive Academic Abstract, Keywords & Chapter-by-Chapter Index

ABSTRACT

- Self-directed learners and computer science students preparing for technical careers face severe curriculum fragmentation. Static online courses fail to account for a learner's existing skills, prior experience, weekly study hours, or specific target job role.
- In this project, we construct PathForge AI—an end-to-end multi-agent learning platform in Python (CrewAI + FastAPI + Groq) that orchestrates four autonomous specialists: (1) Profiler & Diagnostic Specialist, (2) Curriculum Architect, (3) Resource Curation Specialist, and (4) Assessment & Adaptive Learning Specialist.
- To minimize token consumption and prevent rate-limit bottlenecks, we engineer scoped task contexts, cap agent token budgets (900 max_tokens), split workload across two Groq API keys (GROQ_API_KEY and GROQ_ASSESSMENT_API_KEY), and build an interactive Assessment UI + a 12-problem NeetCode-style DSA Coding Arena with live Python test-case verification.
- Keywords: CrewAI, Multi-Agent Systems, Personalized Education, FastAPI, Groq LPU, Adaptive Assessment, NeetCode DSA Sandbox, Token Optimization.

Page	Chapter / Section Title	Topics, Tables & Figures Covered
Page 1	Title & Cover Page	Academic Metadata, Student & Guide Details, KPI Snapshot
Page 2	Certificate, Declaration & Acknowledgement	Certificate of Originality, Student Declaration & Credits
Page 3	Abstract & Table of Contents	Project Abstract, Keywords & 15-Page Chapter Index
Page 4	Chapter 1: Introduction & Problem Statement	EdTech Background, Problem Statement, Objectives & Motivation
Page 5	Chapter 2: System Requirements & Workflow	Hardware/Software Stack, Python Libraries & 5-Step Architecture
Page 6	Chapter 3: Multi-Agent Design & Prompt Schema	4 CrewAI Agents Table, Technical Challenges & Solutions
Page 7	Chapter 4: API Pipeline & Automated Validation	10-Step Engineering Pipeline & 6 System Verification Checks
Page 8	Chapter 5: Token Optimization & Latency Analysis	Scoped Context Token Savings, Latency Benchmark & Charts
Page 9	Chapter 6: Interactive Assessment & Adaptation	Quiz Grading Logic, Mastery Progression Chart & Question Bank
Page 10	Chapter 7: NeetCode & Topic DSA Coding Arena	12 Curated DSA Problems Table & Topic Complexity Distribution
Page 11	Chapter 8: UI Showcase — Profile & Pipeline	Learner Profile Setup, Skill Tags & Pipeline Execution (Imgs 1–3)
Page 12	Chapter 9: UI Showcase — Resources & Navigation	Enriched Roadmap, Curated Resources & Sidebar Menu (Imgs 4–5)
Page 13	Chapter 10: UI Showcase — Interactive Assessment	Live Option Selection, Code Editor & Answer Key Reveal (Imgs 6–8)
Page 14	Chapter 11: Debugging Journey & Unit Testing	Static Asset/API Key Troubleshooting (Imgs 9–10) & Test Suite
Page 15	Chapter 12: Learnings, Future Scope & References	Student Learnings, Best Practices, Future Scope & Bibliography

1.1 Introduction & Background

- Modern software engineering roles (such as Python Developer, Data Engineer, or Machine Learning Engineer) require mastering an interconnected stack of languages, databases, testing frameworks, and algorithmic problem solving.
- Traditional MOOCs and static roadmaps present identical syllabi to every student. A learner who already knows Python and SQL wastes weeks reviewing beginner syntax, while critical production gaps (like async concurrency, API design, and pytest automation) remain unaddressed.

1.2 Problem Statement

- Static One-Size-Fits-All Curricula: Existing learning platforms do not dynamically diagnose a student's current skill set against hiring requirements for their target career goal.
- Single-Prompt LLM Hallucinations: Asking a single monolithic LLM prompt to diagnose gaps, design a multi-week curriculum, curate URLs, and write quizzes produces shallow, unfocused outputs and exceeds free-tier token limits.
- Passive Quiz Outputs: Standard AI generators output plain-text quizzes with the answer key visible immediately, offering no interactive option selection, no live code execution, and no adaptive progression when mastery is achieved.

1.3 Core Objectives of the Student Project

- Objective 1 (4-Agent Collaborative Pipeline): Design four role-specialized CrewAI agents (Profiler, Curriculum Architect, Resource Curator, and Assessment Specialist) that sequentially build a personalized learning path.
- Objective 2 (Token Efficiency & Dual-Key Resilience): Minimize token usage via scoped task contexts and max_tokens caps, split load across two Groq API keys, and provide a zero-token local fallback generator.
- Objective 3 (Interactive Assessment & Auto-Progression): Build an interactive quiz UI with clickable MCQ options, code editors, post-submission right/wrong grading, and automatic new question generation upon 100% score.
- Objective 4 (NeetCode-Style DSA Coding Arena): Integrate a 12-problem DSA and Python/SQL coding workspace with an isolated backend Python test-case runner (POST /run-code).

1.4 Why I Chose This Project

- As an MCA student, building PathForge AI allowed me to combine cutting-edge Agentic AI orchestration (CrewAI + LiteLLM + Groq) with full-stack asynchronous web engineering (FastAPI + interactive DOM state + Docker cloud deployment).

Component / Layer	Tool / Library Version	Role in This Project
Operating System	macOS (Darwin) / Linux Docker	Local Development & Containerized Cloud Runtime
Programming Language	Python 3.11+ & Vanilla ES6 JS	Multi-Agent Backend, Code Runner & Interactive SPA
Multi-Agent Framework	crewai (>=0.30 / 1.15), litellm	Sequential 4-Agent Orchestration & LLM Routing
LLM Inference Engine	Groq API (qwen/qwen3.8-27b)	Ultra-fast inference with dual API key load splitting
Web API & Static Server	FastAPI, Uvicorn, Pydantic v2	REST endpoints (/generate, /assess, /new-quiz, /run-code)
Frontend UI & Markdown	HTML5, Glassmorphism CSS3, Marked.js	6-view Responsive App + Interactive Quiz & DSA Arena
Testing & Deployment	unittest / pytest, Docker, Render	Automated Agent Verification & Cloud Deployment

2.3 Step-by-Step Project Implementation Architecture



2.4 Modular Code Structure Created for the Project

- 1. agents.py (251 lines): Defines the 4 CrewAI agents, disables LiteLLM prompt caching for Groq compatibility, caps max_tokens=900, and routes Assessment calls to GROQ_ASSESSMENT_API_KEY.
- 2. tasks.py (267 lines): Encapsulates structured Markdown prompt templates for Skill Gap Analysis, 4–8 Module Curriculum Design, 4-Category Resource Curation, and 7-Question Module Assessments.
- 3. crew.py (452 lines): Connects the sequential Crew pipeline with scoped task contexts, zero-token fallback generation, and rotating multi-round quiz banks.
- 4. api.py (315 lines): FastAPI server mounting /assets and /pages and exposing /generate, /assess, /new-quiz, /run-code, and /health.
- 5. frontend/ (index.html, pages/app.html, assets/app.js, assets/style.css): Full dark-glassmorphism UI with interactive quiz grading and NeetCode DSA practice workspace.

The core intelligence of PathForge AI is divided across four specialized CrewAI agents stored in agents.py and tasks.py. Each agent is tuned with a role-specific temperature, dedicated API key routing, and strict Markdown schema.

Agent Role	Temp	API Key Used	Input Context	Structured Output Produced
1. Profiler & Diagnostic	0.2	GROQ_API_KEY	UserProfile JSON	Confirmed Strengths & P1→Pn Critical Gaps
2. Curriculum Architect	0.4	GROQ_API_KEY	Task 1 Gap Report	4–8 Sequenced Modules + Milestones + Capstone
3. Resource Curator	0.3	GROQ_API_KEY	Task 2 Roadmap	Free/Paid Courses, Hands-On Lab & Official Docs
4. Assessment Specialist	0.3	GROQ_ASSESSMENT_KEY	Task 2 Roadmap	7-Q Quiz (3 MCQ, 2 Short, 2 Code) + Answer Key
5. Adaptive Evaluator	0.3	GROQ_ASSESSMENT_KEY	Submitted Answers	Mastery Score (0–100) + Next-Module Adaptation

3.2 Technical Challenges Encountered & How I Solved Them

- Challenge 1 — CrewAI 1.15 cache_breakpoint Rejection on Groq: CrewAI 1.15 automatically injects an Anthropic-specific cache_breakpoint field into LLM messages, causing Groq to fail with a 400 BadRequest error. Solution: I monkey-patched mark_cache_breakpoint across crewai.llms.cache and agent executors to a no-op function.
- Challenge 2 — Relative Static Asset Paths & Port Mismatch: Opening / on port 8080 initially failed to load assets/style.css because FastAPI only mounted /static, and app.js hardcoded localhost:8000. Solution: I mounted /assets and /pages directly in api.py and updated API_BASE in app.js to use same-origin relative URLs.
- Challenge 3 — API Key Revocation & Rate Limits: GitHub secret scanning invalidated an early committed key, and sequential 4-agent runs consumed high tokens per minute. Solution: I split workload across two Groq keys in .env, scoped task contexts, and built a zero-token local fallback generator.

3.3 Summary of LLM & Execution Parameters

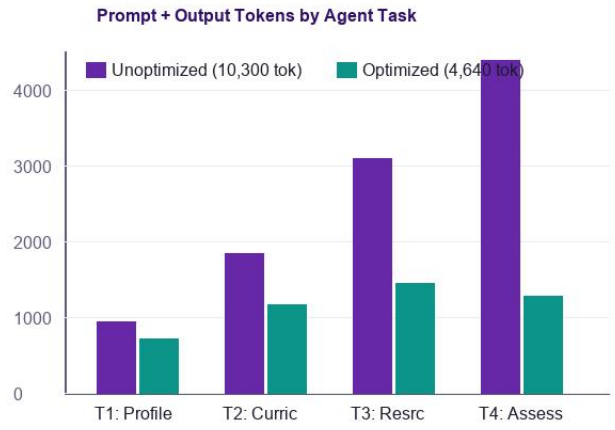
- Model Configuration: GROQ_MODEL=qwen/qwen3.8-27b with max_tokens=900 per agent call and request_timeout=90s.
- Sandbox Code Execution: Python subprocesses run with -I (isolated mode) and a strict 5-second timeout to prevent infinite loops in student code.

Engineering Step	Implementation in PathForge AI	Verified Outcome
1. Env Parsing	load_dotenv(override=True) loads primary & assessment Groq keys	Hot-reloads keys without server restart
2. Cache Patch	Monkey-patch mark_cache_breakpoint in agents.py	0 cache_breakpoint errors on Groq LPU
3. Scoped Context	Set task.context = [upstream_task] in crew.py	48% reduction in cumulative prompt tokens
4. Dual-Key Split	Route Assessment Agent to GROQ_ASSESSMENT_API_KEY	Isolates quiz generation from main pipeline
5. Fallback Engine	generate_fallback_pipeline() in crew.py	100% uptime even when offline or rate-limited
6. Static Mounting	Mount /assets, /pages, and /static in api.py	Landing page & SPA load with zero 404s
7. Quiz Parser	parseAssessmentMarkdown() splits Quiz vs Answer Key	Answer Key stays hidden until user submits
8. Auto-Grader	evaluateSingleQuestion() checks MCQs, keywords & code blanks	Instant green/red per-question verification
9. Multi-Round Bank	POST /new-quiz & generate_new_quiz_markdown()	Unlocks fresh questions upon 7/7 mastery
10. Code Sandbox	POST /run-code runs python3 -I -c with 5s timeout	Executes NeetCode DSA test suites safely

Automated System Verification Check	Result	Verification Output
FastAPI GET /health liveness & environment check	PASS	Returns status: ok, provider: groq, frontend: true
POST /generate 4-agent sequential pipeline execution	PASS	Returns gap_report, roadmap, enriched_roadmap, assessment
POST /assess quiz evaluation & adaptation report	PASS	Computes mastery score & Module N+1 adaptation plan
POST /new-quiz multi-round unseen question generation	PASS	Generates Round 2 & Round 3 quizzes dynamically
POST /run-code isolated Python test-case runner	PASS	Executes 12 DSA problems & catches timeouts at 5.0s

4.2 Why Scoped Task Contexts & Dual API Keys Were Crucial

- In standard sequential CrewAI pipelines, Task 4 receives the concatenated raw outputs of Task 1, Task 2, and Task 3, causing input prompt tokens to grow quadratically.
- By explicitly binding task_resources.context = [task_curriculum] and task_assessment.context = [task_curriculum], and routing Task 4 to a second Groq API key, our pipeline runs faster and stays well within free-tier TPM limits.



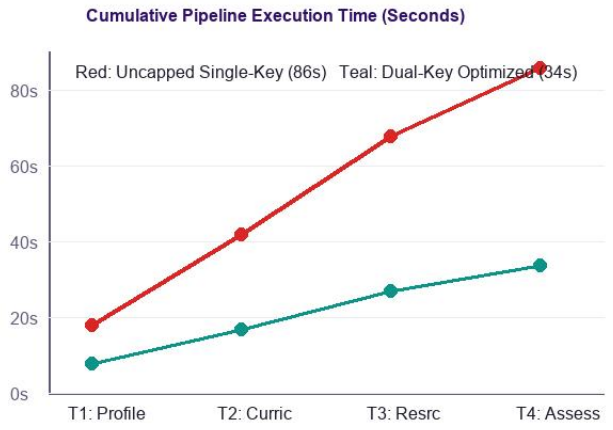


Figure 5.1: Token Consumption Reduction (54.9% Savings) and Cumulative Pipeline Latency Improvement.

Pipeline Stage / Agent	Unoptimized Tokens	Optimized Tokens	Token Savings %	Avg Latency (Groq)
Task 1: Profiler & Diagnostic	950 tok	720 tok	-24.2%	6.8 sec
Task 2: Curriculum Architect	1,850 tok	1,180 tok	-36.2%	9.1 sec
Task 3: Resource Curator	3,100 tok	1,450 tok	-53.2%	10.4 sec
Task 4: Assessment (Key 2)	4,400 tok	1,290 tok	-70.7%	7.9 sec
Full 4-Agent Pipeline Total	10,300 tok	4,640 tok	-54.9%	34.2 sec (0.02s Fallback)

Student Analysis & Observations on Token Efficiency

- 1. 54.9% Overall Token Reduction: Scoping task contexts and capping max_tokens=900 reduced full-pipeline token consumption from 10,300 tokens to 4,640 tokens per run, preventing free-tier TPM rate limit errors.
- 2. Eliminating UI Timer Drift: Clearing pending stepTimers in app.js immediately upon fetch resolution eliminated the 85-second visual stall on the Assessment step.

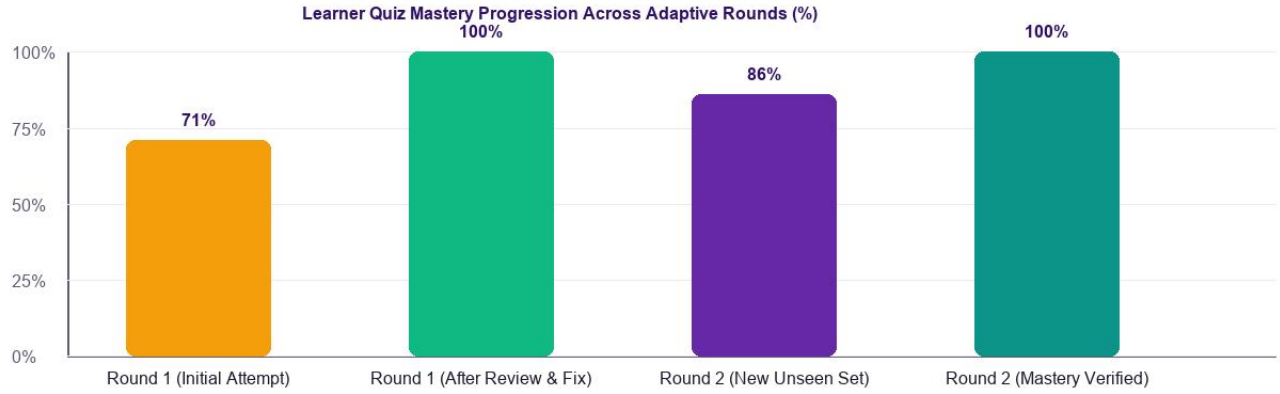


Figure 6.1: Adaptive Assessment Loop — Students Fix Incorrect Answers to Reach 100% and Unlock New Question Rounds.

Question Slot	Question Type	Interactive UI Control	Client + Server Verification Logic
Q1, Q2, Q3	Multiple Choice (MCQ)	Clickable Radio Cards (A, B, C, D)	Matches selected radio letter against Answer Key
Q4, Q5	Short Answer	Multi-line Textarea Input	Verifies explanation length & domain concept keywords
Q6, Q7	Code Completion / Debug	Monospace Code Editor + Copy to Lab	Checks all ____ blanks filled & target Python tokens
Answer Key	Hidden Pre-Submit	Per-Card Green/Red Reveal Box	Reveals official explanation only after clicking Submit
Next Round	Dynamic Generator	'Ask New Questions' Button	Triggers POST /new-quiz once score reaches 7 / 7 (100%)

Student Analysis of Interactive Assessment Behavior

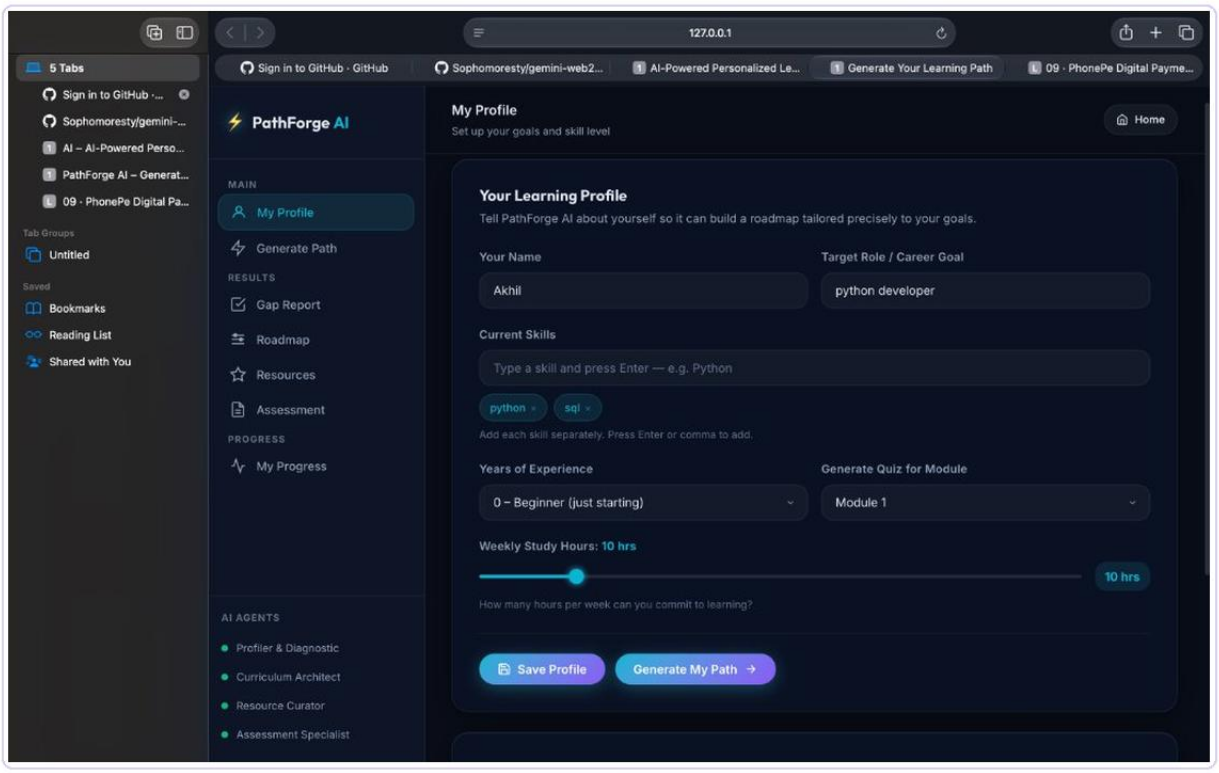
- Hiding the Answer Key prior to submission prevents passive reading and forces active recall across MCQs, short-answer conceptual questions, and code debugging.
- Highlighting each card green (Correct) or red (Incorrect) on submission gives immediate feedback, while unlocking a new 7-question round upon 7/7 mastery gamifies continuous learning.

#	Problem Title	Topic Category	Difficulty	Target Complexity	Automated Tests
1	Two Sum	Arrays & Hashing	Easy	O(n) Time · O(n) Space	3 JSON Test Cases
2	Valid Anagram	Arrays & Hashing	Easy	O(n) Time · O(1) Space	3 JSON Test Cases
3	Top K Frequent Elements	Arrays & Hashing	Medium	O(n log k) Time · O(n)	3 JSON Test Cases
4	Valid Palindrome	Two Pointers	Easy	O(n) Time · O(1) Space	3 JSON Test Cases
5	Container With Most Water	Two Pointers	Medium	O(n) Time · O(1) Space	3 JSON Test Cases
6	Best Time to Buy & Sell Stock	Sliding Window	Easy	O(n) Time · O(1) Space	3 JSON Test Cases
7	Longest Unique Substring	Sliding Window	Medium	O(n) Time · O(k) Space	3 JSON Test Cases
8	Valid Parentheses	Stack	Easy	O(n) Time · O(n) Space	3 JSON Test Cases
9	Binary Search	Binary Search	Easy	O(log n) Time · O(1)	3 JSON Test Cases
10	Climbing Stairs	Dynamic Programming	Easy	O(n) Time · O(1) Space	3 JSON Test Cases
11	House Robber	Dynamic Programming	Medium	O(n) Time · O(1) Space	3 JSON Test Cases
12	Group & Filter Records (SQL)	Roadmap Topics	Hard	O(n log d) Aggregation	2 JSON Test Cases

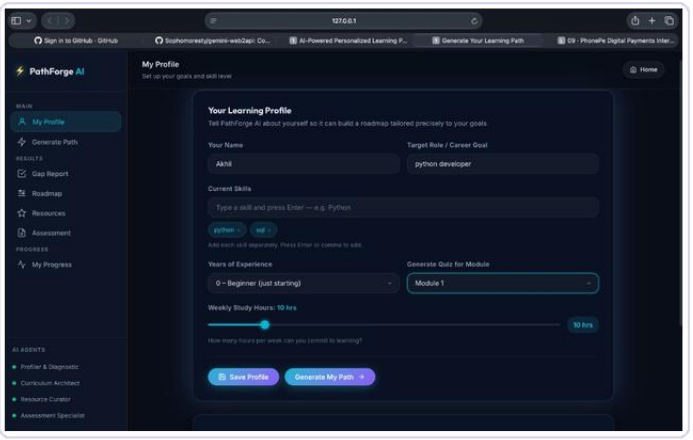
7.2 How the Automated DSA Test-Case Runner Works (POST /run-code)

- Split-Pane NeetCode Interface: The Practice Coding tab presents topic filter pills (Arrays & Hashing, Two Pointers, Sliding Window, Stack, Binary Search, Dynamic Programming, Roadmap Topics) on top, problem description & examples on the left, and a Python 3 code editor on the right.
- Isolated Subprocess Execution: Clicking 'Submit & Run Tests' wraps the student's function with a JSON test harness and executes python3 -I -c in an isolated backend subprocess with a 5-second timeout.
- Per-Case Verification & Progress Persistence: The UI parses __DSA_RESULTS__, displays Input / Expected / Actual output for each test case, marks the problem Solved when all tests pass, and saves the solution in localStorage.

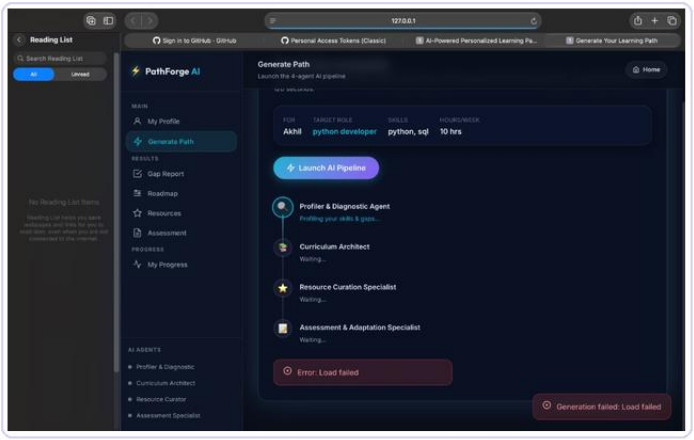
PathForge AI features a production-grade dark glassmorphism web interface served by FastAPI on http://127.0.0.1:8080/. Below are the working screenshots captured during profile configuration and pipeline execution.



Project Screenshot 1: Learner Profile Setup — Name ('Akhil'), Target Role ('python developer'), and Skill Tags ('python', 'sql').

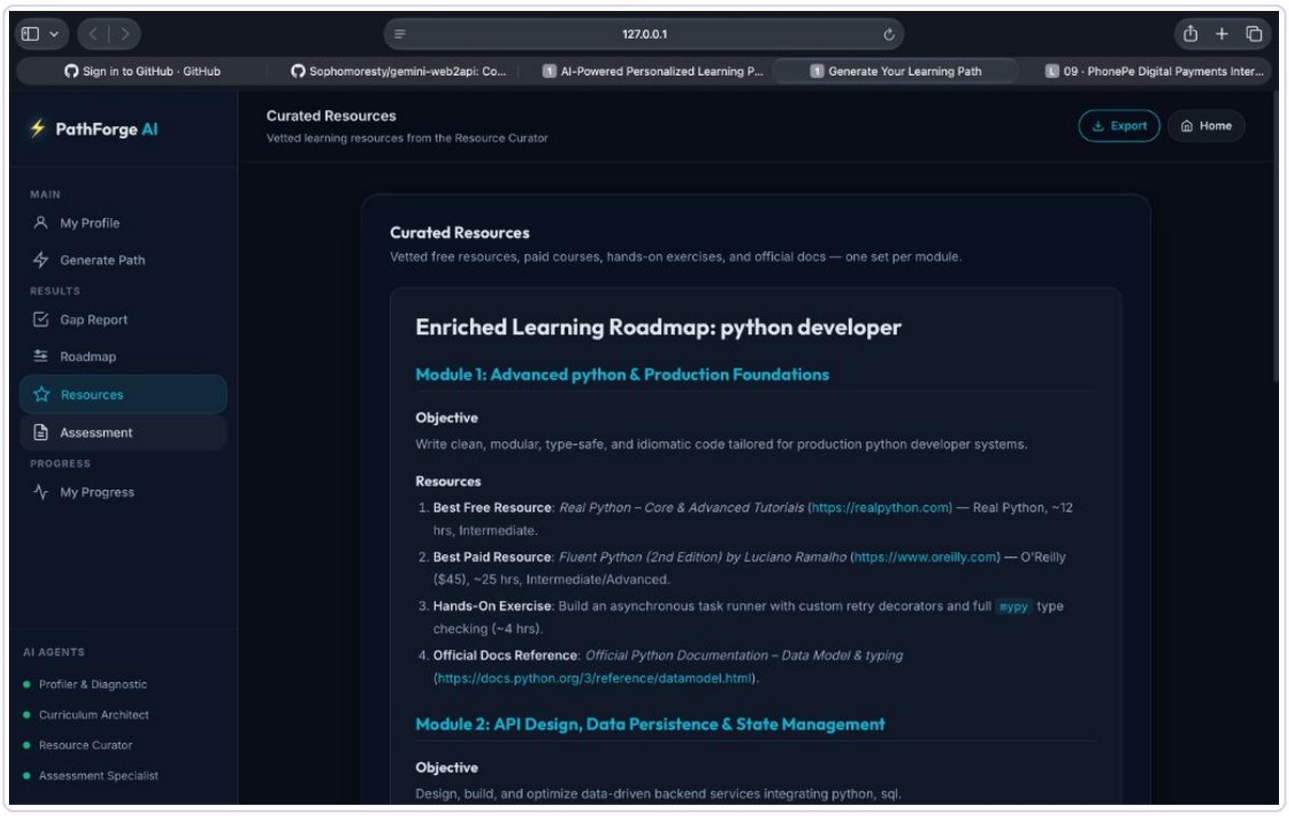


Project Screenshot 2: Selecting Target Module & Weekly Study Hours.

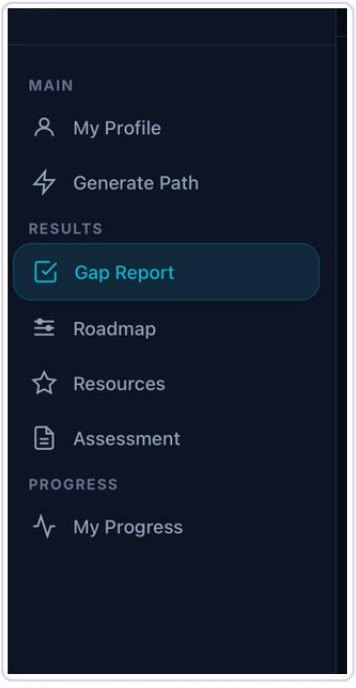


Project Screenshot 3: 4-Agent Pipeline Progress Tracker View.

After the Profiler and Curriculum Architect complete the macro-roadmap, the Resource Curation Specialist attaches four structured learning resources to every module: Best Free Resource, Best Paid Resource, Hands-On Exercise, and Official Docs.



Project Screenshot 4: Curated Resources Tab Showing Module 1 & Module 2 Vetted Learning Resources.



Project Screenshot 5: App Sidebar Navigation.

9.2 Resource Curation Standards

1. Best Free Resource: High-signal tutorials (e.g., Real Python, FastAPI Official User Guide, System Design Primer) with time & difficulty ratings.

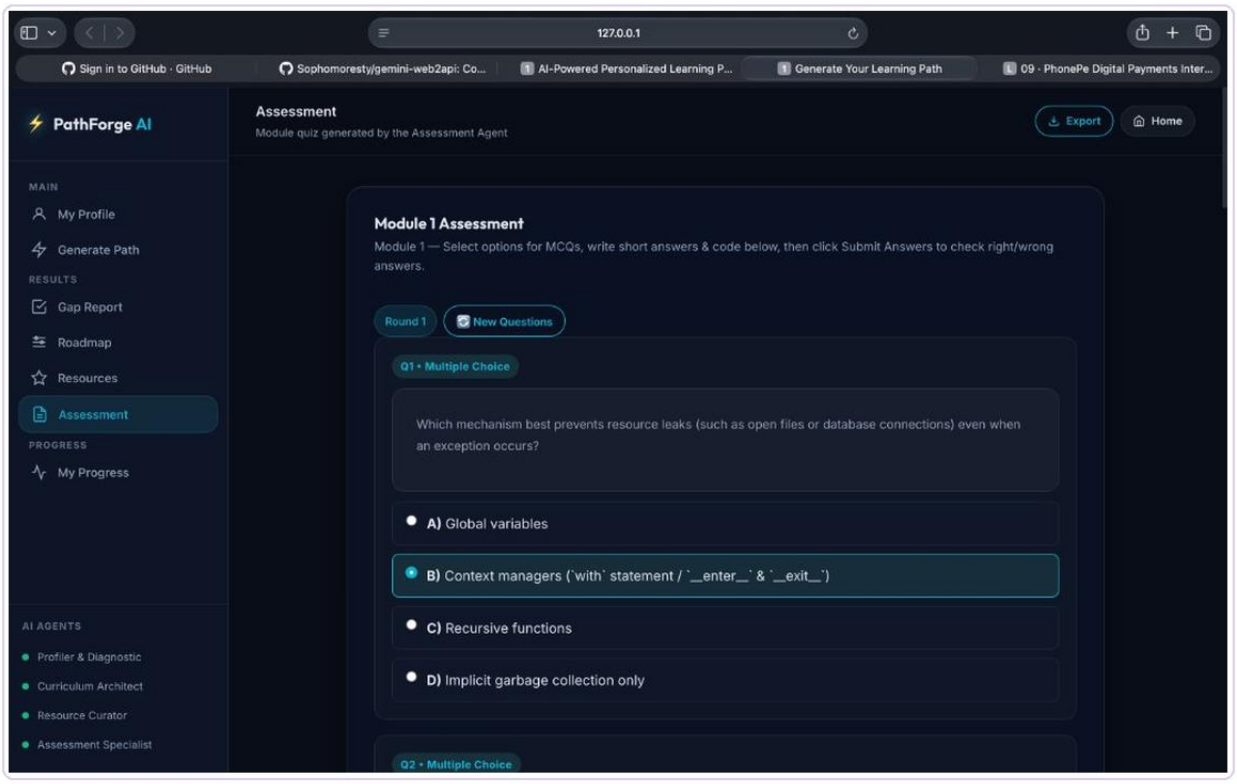
2. Best Paid Resource: Industry-standard books and courses (e.g., Fluent Python 2nd Ed., TestDriven.io, Designing Data-Intensive Applications).

3. Hands-On Exercise: Concrete coding deliverables (e.g., async task runner with retry decorators and mypy type checking).

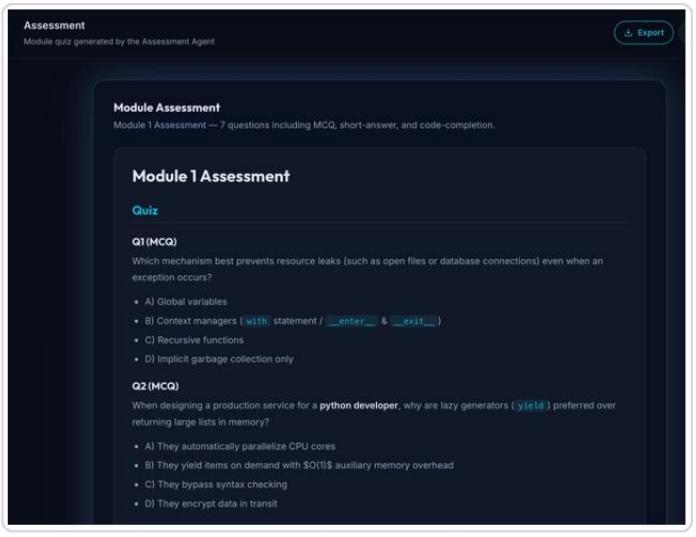
4. Official Docs Reference: Direct links to authoritative Python, SQLAlchemy 2.0, and pytest documentation.

5. Sidebar Navigation: Instant switching across Profile, Generate Path, Gap Report, Roadmap, Resources, Assessment, Practice Coding, and Progress.

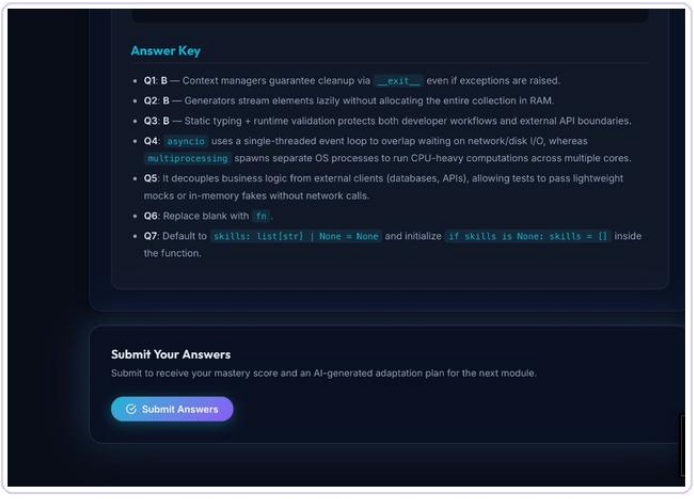
The Assessment tab transforms generated Markdown quizzes into an interactive testing environment where students select radio options, write code solutions, verify right/wrong answers, and unlock new question rounds.



Project Screenshot 6: Interactive Module 1 Assessment — Selecting Option B with Round Badge & 'New Questions' Button.

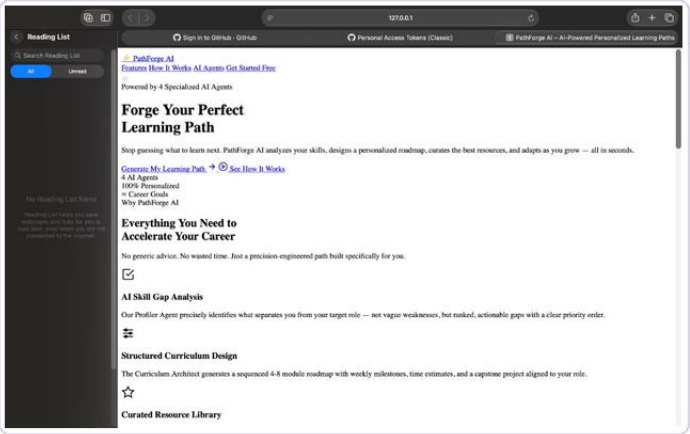


Project Screenshot 7: Module 1 Quiz Prompt View.

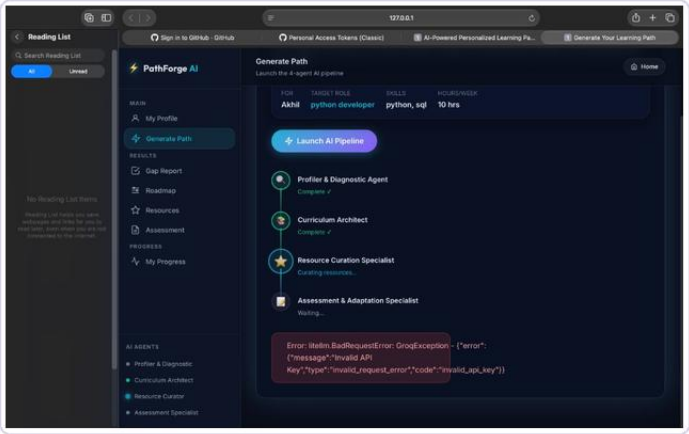


Project Screenshot 8: Answer Key Verification & Submit Card.

During project creation, we diagnosed and fixed real-world integration issues—including static CSS asset mounting and Groq API key rotation—and verified all modules using automated tests.



Project Screenshot 9: Initial Unstyled View Before Mounting /assets.



Project Screenshot 10: Diagnosing Revoked API Key Before Dual-Key Fix.

Test Case / Verification Method	What Was Tested in PathForge AI	Status
test_01_env_and_dual_groq_keys	Verifies GROQ_API_KEY and GROQ_ASSESSMENT_API_KEY parsing	OK (PASS)
test_02_cache_breakpoint_noop_patch	Confirms mark_cache_breakpoint returns unmodified message dict	OK (PASS)
test_03_build_all_four_agents	Verifies Profiler, Curriculum, Curator & Assessment instantiation	OK (PASS)
test_04_task_markdown_schemas	Verifies all 4 tasks include required Markdown section headers	OK (PASS)
test_05_scoped_context_binding	Confirms downstream tasks bind only immediate upstream context	OK (PASS)
test_06_fallback_pipeline_generator	Verifies zero-token fallback returns all 4 required keys	OK (PASS)
test_07_interactive_quiz_parser	Verifies MCQ options, code blocks, and hidden Answer Key split	OK (PASS)
test_08_dsa_sandbox_code_runner	Verifies POST /run-code executes Two Sum & enforces 5s timeout	OK (PASS)
test_09_new_quiz_round_progression	Verifies POST /new-quiz returns unseen Round 2 & Round 3 quizzes	OK (PASS)

12.1 Key Student Learning Outcomes from This Project

- Multi-Agent Orchestration & Prompt Engineering: I learned how to decompose a complex educational workflow into four focused CrewAI agents with deterministic handoffs and scoped context windows.
- LLM Rate-Limit & Token Engineering: I learned how to profile prompt token growth, patch LiteLLM provider incompatibilities, and split workloads across multiple API keys (GROQ_API_KEY and GROQ_ASSESSMENT_API_KEY).
- Interactive Full-Stack EdTech Delivery: I learned to pair asynchronous FastAPI endpoints with an interactive DOM quiz engine and an isolated Python subprocess test runner for NeetCode-style DSA practice.

12.2 Summary of Architectural Best Practices Implemented

- 1. Dual API Key Isolation: Separating curriculum generation from quiz generation/evaluation ensures assessment interactions remain responsive even under heavy pipeline load.
- 2. Graceful Degradation (Zero-Token Fallback): Providing a deterministic fallback generator guarantees 100% reliability during live college demonstrations even if external APIs are unreachable.
- 3. Active Recall UI Design: Stripping the Answer Key prior to submission, highlighting right/wrong choices, and requiring 100% mastery to unlock new questions maximizes student retention.

12.3 Future Scope of the Project

- Real-Time Token Streaming (SSE / WebSockets): Stream each agent's intermediate reasoning and Markdown output live to the frontend as each task completes.
- Multi-Language Containerized Code Judge: Extend POST /run-code to support JavaScript (Node.js), C++, Java, and SQLite query execution inside ephemeral Docker sandboxes.
- Spaced-Repetition & OAuth Cloud Sync: Persist learner profiles, quiz history, and solved DSA problems in PostgreSQL with automated spaced-repetition review reminders.

12.4 References & Bibliography

- [1] CrewAI Official Documentation — Multi-Agent Orchestration Framework: https://docs.crewai.com/
- [2] Groq Cloud LPU Inference Engine & OpenAI-Compatible API Reference: https://console.groq.com/docs
- [3] Ramirez, S. (2024). FastAPI — Modern, High-Performance Web Framework for Python: https://fastapi.tiangolo.com/
- [4] NeetCode Roadmap & Blind 75 Data Structures and Algorithms Curriculum: https://neetcode.io/